

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 12: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 17 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

- A1.** Define a *polynomial-time reduction* $X \leq_p Y$. State precisely the two consequences of $X \leq_p Y$ regarding the tractability of X and Y , and explain which of the two is used to *prove a problem hard*.
- A2.** Explain why the theory of NP-completeness restricts attention to *decision problems* (yes/no answers). Show how the optimization problem “find a maximum independent set” can be solved using a polynomial number of calls to the decision problem “does G have an independent set of size $\geq k$?”.
- A3.** Define the class P and the class NP in terms of algorithms, certificates, and verifiers. In one sentence each, contrast what P and NP capture.
- A4.** Prove that $P \subseteq NP$.
- A5.** For the INDEPENDENT SET problem, describe explicitly: (a) what a certificate is, (b) what the verifier checks, and (c) why the verifier runs in polynomial time. Do the same for 3-SAT.
- A6.** State precisely what it means for a problem to be *NP-hard* and what it means to be *NP-complete*. Why does NP-completeness require a separate proof that the problem is *in* NP, on top of the hardness argument?
- A7.** State the Cook–Levin theorem. In two or three sentences, describe the high-level idea of why *every* problem in NP reduces to SAT.
- A8.** Prove that $\leq_p$ is transitive: if $X \leq_p Y$ and $Y \leq_p Z$ then $X \leq_p Z$. Then explain how transitivity, together with one known NP-complete problem, makes subsequent NP-completeness proofs much shorter.
- A9.** State the standard “recipe” (the list of steps) for proving that a problem Y is NP-complete. Identify which step establishes membership in NP and which establishes NP-hardness.

- A10.** Prove the complementarity lemma: in any graph $G = (V, E)$ with $|V| = n$, a set S is an independent set iff $V \setminus S$ is a vertex cover. Deduce that G has an independent set of size $\geq k$ iff it has a vertex cover of size $\leq n - k$, and hence that $\text{INDEPENDENT SET} \leq_p \text{VERTEX COVER}$.
- A11.** Prove that S is an independent set in G iff S is a clique in the complement graph $\overline{G}$, and conclude that $\text{INDEPENDENT SET} \leq_p \text{CLIQUE}$.
- A12.** Describe the reduction $3\text{-SAT} \leq_p \text{INDEPENDENT SET}$: how many vertices does the constructed graph have, what are the “triangle” edges, what are the “conflict” edges, and what is the target size k ? Prove the forward direction (a satisfying assignment yields an independent set of size k).
- A13.** For the same reduction, prove the reverse direction: an independent set of size k yields a satisfying assignment. In particular, explain why such an independent set must contain exactly one vertex from each clause-triangle, and why the conflict edges guarantee the resulting assignment is consistent.
- A14.** Sketch the reduction $\text{HAMILTONIAN CYCLE} \leq_p \text{TSP}$ (decision version). What distances do you assign to edges and non-edges of the graph, what tour-length bound D do you use, and why does a tour of length $\leq D$ correspond exactly to a Hamiltonian cycle?
- A15.** SUBSET SUM has a dynamic-programming algorithm running in time $O(nW)$, where W is the target. Explain why this does *not* contradict the NP-completeness of SUBSET SUM . Define *pseudo-polynomial* time and explain the distinction between the numeric value W and the input size.
- A16.** A student writes: “To prove VERTEX COVER is NP-hard, I reduced VERTEX COVER to 3-SAT ; since 3-SAT is NP-complete, so is VERTEX COVER .” Identify the error precisely, and state what reduction the student should have constructed instead.
- A17.** *Discussion (open-ended).* You are handed a brand-new optimization problem at work and asked whether there is an efficient algorithm. Describe a systematic process for (a) deciding whether to search for a polynomial-time algorithm or for an NP-completeness proof, and (b) if you suspect hardness, how you would go about constructing a reduction. What known NP-complete problems would you try first for a graph problem? a packing/number problem? an ordering/routing problem?

Part B: Quiz

- B1. (True/False)** If $X \leq_p Y$ and Y has a polynomial-time algorithm, then X has a polynomial-time algorithm.
- B2. (Multiple choice)** The class NP is the set of decision problems for which:
- (a) a solution can be found in polynomial time
 - (b) a proposed solution (certificate) can be verified in polynomial time
 - (c) no polynomial-time algorithm can possibly exist
 - (d) the answer is always “yes”
- B3. (True/False)** Proving $P = NP$ would follow from finding a single polynomial-time algorithm for any one NP-complete problem.
- B4. (Multiple choice)** The Cook–Levin theorem establishes that:
- (a) every problem in NP is in P
 - (b) SAT (and 3-SAT) is NP-complete
 - (c) SUBSET SUM can be solved in polynomial time
 - (d) $P \neq NP$
- B5. (True/False)** To prove a problem Y is NP-hard, it suffices to give a polynomial-time reduction *from* a known NP-complete problem *to* Y .
- B6. (Short answer)** In a graph G on n vertices, if the maximum independent set has size 13, what is the size of the minimum vertex cover (in terms of n)?
- B7. (Multiple choice)** In the reduction $3\text{-SAT} \leq_p \text{INDEPENDENT SET}$, a 3-SAT formula with k clauses produces a graph in which we look for an independent set of size:
- (a) 1
 - (b) 3
 - (c) k
 - (d) $3k$
- B8. (True/False)** A set S is a clique in G if and only if S is an independent set in the complement graph $\overline{G}$.
- B9. (Short answer)** 2-COLORING is in P but 3-COLORING is NP-complete. What classical polynomial-time test decides 2-colorability?
- B10. (Multiple choice)** An $O(nW)$ algorithm for SUBSET SUM (with target W) is best described as:
- (a) polynomial in the input size
 - (b) pseudo-polynomial (polynomial in the numeric value W , not in $\log W$)
 - (c) proof that $P = NP$
 - (d) exponential in n for all inputs

Part C: Solutions

Part A

- A1.** $X \leq_p Y$ means an arbitrary instance of X can be solved using a polynomial number of basic steps plus a polynomial number of calls to an oracle (subroutine) for Y . The two consequences: (a) if $Y \in P$ then $X \in P$ (tractability flows forward); (b) if $X \notin P$ then $Y \notin P$ (intractability flows backward). To *prove a problem Y hard*, we use (b): we take a known-hard X and show $X \leq_p Y$, so Y is at least as hard as X .
- A2.** Decision problems have a single bit of output, which makes “polynomial time” clean to define and lets reductions transform yes/no answers into yes/no answers. The restriction loses nothing essential because optimization reduces to decision: to find the size of a maximum independent set in G on n vertices, ask the decision oracle “is there an independent set of size $\geq k$?” for $k = n, n-1, \dots$ (or binary-search on k in $O(\log n)$ queries) to find the largest feasible k^* . To recover an actual set, fix vertices one at a time: for each vertex v , delete it and ask whether the remaining graph still has an independent set of size k^* ; keep v out (it was not needed) if so, otherwise it belongs to the set – a polynomial number of oracle calls in total.
- A3.** P is the set of decision problems solvable by an algorithm that always halts in time polynomial in the input length. NP is the set of decision problems that have an *efficient verifier*: a polynomial-time algorithm $B(s, t)$ such that s is a yes-instance iff there exists a polynomial-length certificate t with $B(s, t) = \text{yes}$. In one sentence each: P captures problems we can efficiently *solve*; NP captures problems whose solutions we can efficiently *check* once handed a candidate.
- A4.** Let $X \in P$ via a polynomial-time algorithm A . Define a verifier $B(s, t) := A(s)$ that ignores the certificate t (take t to be the empty string). Then B runs in polynomial time, and $X(s) = \text{yes}$ iff there exists a certificate (trivially, the empty one) with $B(s, t) = \text{yes}$, namely exactly when $A(s) = \text{yes}$. Hence $X \in NP$, so $P \subseteq NP$.
- A5.** INDEPENDENT SET, input (G, k) : (a) the certificate is a set S of vertices; (b) the verifier checks $|S| \geq k$ and that no edge of G has both endpoints in S ; (c) this takes $O(|S|^2)$ or $O(|E|)$ time – polynomial. 3-SAT, input a CNF formula Φ : (a) the certificate is a truth assignment to the variables; (b) the verifier evaluates each clause under the assignment and checks every clause is satisfied; (c) this is linear in the size of Φ . In both cases the certificate has length polynomial in the input and the check is polynomial, so both problems are in NP .
- A6.** Y is *NP-hard* if $X \leq_p Y$ for every $X \in NP$ (it is at least as hard as everything in NP). Y is *NP-complete* if it is NP-hard *and* $Y \in NP$. We separately prove membership in NP because NP-hardness alone does not place Y inside NP – there are NP-hard problems (e.g. certain undecidable or PSPACE-hard problems) that are not in NP at all. “Complete” means hardest *within* the class, which requires being a member of the class.
- A7. Cook–Levin theorem:** SAT (and 3-SAT) is NP-complete. High-level idea: take any $X \in NP$ with efficient verifier B running in polynomial time $p(|s|)$. The entire computation of B on input s and certificate t – the memory contents at each of the $p(|s|)$ time steps – can be encoded with polynomially many Boolean variables, and a polynomial-size CNF formula can express “these variables describe a valid accepting computation of B for some certificate t .” That formula is satisfiable iff a valid certificate exists iff s is a yes-instance, giving $X \leq_p \text{SAT}$.

- A8. Transitivity.** Suppose $X \leq_p Y$ and $Y \leq_p Z$. Solve an instance of X using the first reduction: polynomially many steps and polynomially many calls to a Y -oracle. Replace each Y -oracle call by the second reduction, which solves Y using polynomially many steps and polynomially many calls to a Z -oracle. Composing, X is solved with a polynomial number of Z -oracle calls and polynomial overhead, so $X \leq_p Z$. *Why it shortens proofs:* once a single problem (SAT, via Cook–Levin) is known to be NP-complete, to show a new Y is NP-hard we need only exhibit *one* reduction from *one* known NP-complete problem X to Y ; transitivity then chains it back through X to *every* problem in NP, so we never have to re-encode arbitrary NP computations.
- A9. Recipe to prove Y NP-complete:** (1) state Y as a decision problem; (2) prove $Y \in \text{NP}$ by giving a short certificate and a polynomial-time verifier [*this is the membership step*]; (3) pick a known NP-complete problem X ; (4) construct a polynomial-time reduction $X \leq_p Y$ (the gadget); (5) prove correctness in both directions ($x \text{ yes} \iff f(x) \text{ yes}$); (6) check the reduction is polynomial-time [*steps 3–6 establish NP-hardness*]. Membership (step 2) plus NP-hardness (steps 3–6) together give NP-completeness.
- A10. Complementarity.** ($\Rightarrow$) Let S be independent. For any edge $(u, v) \in E$, u and v are not both in S (independence), so at least one is in $V \setminus S$ – hence $V \setminus S$ covers every edge, i.e. it is a vertex cover. ($\Leftarrow$) Let $V \setminus S$ be a vertex cover. If some edge had both endpoints in S , that edge would be uncovered, a contradiction; so no edge lies inside S , i.e. S is independent. Since $|V \setminus S| = n - |S|$, an independent set of size $\geq k$ corresponds exactly to a vertex cover of size $\leq n - k$. This gives the reduction $\text{INDEPENDENT SET} \leq_p \text{VERTEX COVER}$: map $(G, k) \mapsto (G, n - k)$ (constant work plus one oracle call), preserving the answer.
- A11.** S is independent in $G \iff$ no pair of vertices in S is an edge of $G \iff$ every pair of vertices in S is a *non-edge* of $G \iff$ every pair in S is an edge of $\overline{G}$ (whose edges are precisely the non-edges of G) $\iff S$ is a clique in $\overline{G}$. The reduction $\text{INDEPENDENT SET} \leq_p \text{CLIQUE}$ maps $(G, k) \mapsto (\overline{G}, k)$; building $\overline{G}$ takes $O(n^2)$ time, and a k -independent set in G exists iff a k -clique in $\overline{G}$ exists.
- A12. Construction.** For a formula with k clauses, build a graph with $3k$ vertices: three vertices per clause, one per literal. *Triangle edges:* within each clause, join all three of its literal-vertices pairwise (forming a triangle). *Conflict edges:* between any two vertices in different clauses whose literals are complementary (one is x_i , the other $\overline{x_i}$), add an edge. Target size k = number of clauses.
- Forward direction.** Suppose Φ has a satisfying assignment. Each clause has at least one literal evaluating to true; pick one such literal per clause and take its vertex. This selects exactly one vertex from each triangle (so no triangle edge is violated, since triangle edges only connect vertices *within* a clause and we picked one), and no two selected vertices conflict: a conflict edge connects x_i and $\overline{x_i}$, but under a single assignment these literals cannot both be true, so we never selected both. Hence the selected k vertices form an independent set of size k .
- A13. Reverse direction.** Suppose G has an independent set S with $|S| = k$. Within each clause-triangle the three vertices are mutually adjacent, so S contains at most one vertex per triangle. There are exactly k triangles and $|S| = k$, so S contains *exactly one* vertex per triangle. Construct an assignment by setting, for each vertex in S labelled with literal ℓ , the underlying variable so that ℓ is true (e.g. if $\ell = \overline{x_i}$ set x_i false). This is well-defined and consistent:

if S contained both an x_i -vertex and an $\bar{x}_i$ -vertex, the conflict edge between them would violate independence – but S is independent, so no such pair exists, and we never demand contradictory values of any variable. Unconstrained variables are set arbitrarily. Since S has a vertex in every clause-triangle and that vertex's literal is made true, every clause is satisfied, so Φ is satisfiable.

- A14. Reduction Hamiltonian Cycle $\leq_p$ TSP.** Given a graph $G = (V, E)$ with $|V| = n$, create one city per vertex. Set the distance $d(i, j) = 1$ if $(i, j) \in E$ and $d(i, j) = 2$ (any value > 1 , e.g. ∞) if $(i, j) \notin E$. Set the bound $D = n$. A tour visits all n cities and returns to start, using exactly n edges; its length is $\geq n$, with equality iff *all* n tour-edges have distance 1, i.e. all are edges of G . So a tour of length $\leq n$ exists iff there is a cycle through all vertices using only edges of G – precisely a Hamiltonian cycle. The construction is $O(n^2)$, hence polynomial, and the equivalence is exact.
- A15.** The $O(nW)$ algorithm is *pseudo-polynomial*: it is polynomial in n and in the numeric *value* W , but the *input size* is the number of bits used to write the numbers, roughly $\sum_i \log w_i + \log W$, i.e. proportional to $\log W$, not W . When $W \approx 2^m$ for input size $\approx m$ bits, $O(nW) = O(n \cdot 2^m)$ is exponential in the input size. So the DP is fast only when the numbers are small (written in unary, or bounded by a polynomial in n); for the general problem, with numbers given in binary, it is exponential. This is fully consistent with NP-completeness, which is about polynomiality in the input *size*.
- A16.** The error is the *direction* of the reduction. The student reduced VERTEX COVER $\leq_p$ 3-SAT, which only shows VERTEX COVER is no *harder* than 3-SAT (i.e. VERTEX COVER $\in$ NP, which we already knew); it says nothing about VERTEX COVER being hard. To prove VERTEX COVER NP-hard, the student must reduce a known NP-complete problem *to* VERTEX COVER, e.g. 3-SAT $\leq_p$ VERTEX COVER (or INDEPENDENT SET $\leq_p$ VERTEX COVER via complementarity). Hardness flows *from* the known problem *into* the target, so the known-hard problem must be the source of the arrow.
- A17.** A systematic process. **(a) Decide which way to bet.** First nail down the exact decision version and the input encoding. Try small instances by hand and look for structure that matches a paradigm you know (greedy, DP, flow, matroid) – if a natural subproblem structure or exchange argument appears, pursue a polynomial algorithm. Simultaneously, look for “combinatorial explosion” signals: a need to choose a subset/permutation/assignment with global interacting constraints, the appearance of words like “exactly,” “cover,” “Hamiltonian,” “3-coloring,” or an obvious resemblance to a known NP-complete problem. If polynomial attempts keep failing and the problem smells like SAT/packing/routing, switch to seeking an NP-completeness proof. **(b) Build the reduction.** Identify the source problem whose “logic” your problem can encode, design gadgets that translate that problem’s variables/constraints into your problem’s objects, and verify both directions. **First problems to try:** for a *graph* problem – INDEPENDENT SET, VERTEX COVER, CLIQUE, 3-COLORING, or HAMILTONIAN CYCLE; for a *packing/number* problem – SUBSET SUM or KNAPSACK; for an *ordering/routing* problem – HAMILTONIAN CYCLE or TSP; and 3-SAT as the universal fallback when the structure is logical/constraint-based.

Part B

- B1. True.** This is the forward direction of a reduction: tractability of Y transfers to X (Proposition: $X \leq_p Y$ and $Y \in P$ imply $X \in P$).

- B2. (b)** A proposed solution (certificate) can be verified in polynomial time. This is the verifier-based definition of NP; it does *not* require that solutions can be *found* quickly.
- B3. True.** If any NP-complete problem is in P, then since every problem in NP reduces to it, all of NP is in P, so $P = NP$.
- B4. (b)** SAT (and 3-SAT) is NP-complete. Cook–Levin provides the first NP-complete problem; it does not resolve P vs. NP.
- B5. True.** NP-hardness of Y is exactly: some known NP-complete X satisfies $X \leq_p Y$ (and by transitivity all of NP reduces to Y). The arrow goes *from* the known-hard problem *to* Y .
- B6.** The minimum vertex cover has size $n - 13$, by complementarity ($|\min VC| = n - |\max IS|$).
- B7. (c) k .** The constructed graph has $3k$ vertices (a triangle per clause), but the target independent-set size is k – one vertex per clause.
- B8. True.** A clique in G is a set in which all pairs are edges of G , i.e. no pair is an edge of $\overline{G}$, i.e. an independent set in $\overline{G}$ (and vice versa).
- B9.** Check whether the graph is **bipartite** (no odd cycle), via BFS/DFS 2-coloring. A graph is 2-colorable iff it is bipartite, which is decidable in linear time.
- B10. (b)** Pseudo-polynomial: polynomial in the numeric value W but exponential in the input size $\log W$ when W is given in binary – consistent with SUBSET SUM being NP-complete.